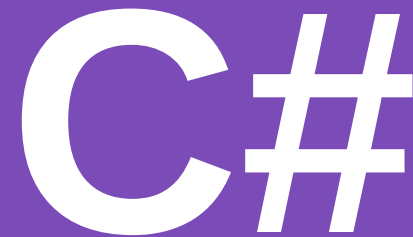


C# tanulási út

Az alapoktól az osztályokig – fájlkezeléssel

Magyar nyelvű jegyzet kezdőknek

Közérthető magyarázatok • működő példakódok • gyakorló feladatok

The C# logo is displayed in white text. It consists of a large 'C' followed by a '#' symbol. The logo is positioned on the right side of the slide, overlaid on a large purple circle. A smaller, darker purple circle is partially visible behind it to the right.

1

Alapok

Első program,
kiírás, beolvasás

2

Változók és típusok

int, string,
konverzió, szöveg

3

Döntések és ciklusok

if, switch, for, while

4

Tömbök és listák

Tömb, List, foreach

5

Metódusok, hibakezelés

Függvények,
try/catch

6

Fájlkezelés

Olvasás, írás,
mentés

7

Osztályok

Objektumok,
öröklődés

Hogyan tanulj ebből? Minden példát gépelj be kézzel Visual Studióban, futtasd, aztán változtass rajta valamit. Minden téma után egy „**Mélyebben**” dia következik: részletes magyarázat és **íratlan szabályok**. A végén a **mini-feladatokat** próbáld meg segítség nélkül megoldani.

- A **C#** (ejtsd: szí-sárp) a Microsoft programozási nyelve. Készíthetsz vele játékot (Unity), weboldalt, asztali programot, mobilappot.
- Amit leírsz, azt a **fordító** (compiler) alakítja géppel érthető programmá. Ez a **.NET** platformon fut.
- Eszköz: **Visual Studio Community** (ingyenes). Ez az írószerkesztő, fordító és hibakereső egyben.
- A munka menete: **kód írása** → **Build (Ctrl+Shift+B)** → **futtatás (F5)**.

Tipp: a **Ctrl+F5** debug nélkül futtat, és a konzolablak a végén nyitva marad, így el tudod olvasni a kimenetet.

RÉSZLETES MAGYARÁZAT

- A C# **magas szintű** nyelv: te emberközelí kódot írsz, a **fordító** (compiler) ezt köztes nyelvre (IL) fordítja, a .NET futtatókörnyezet pedig futás közben gépi kóddá alakítja.
- Ezért jelez a Visual Studio már **írás közben** hibát (piros hullámos vonal): a fordító nem tud lefordítani olyat, amit nem ért.
- A **Build** csak lefordít, az **F5** lefordít és elindít hibakeresővel (debugger), a **Ctrl+F5** hibakereső nélkül indít.
- Kétféle hiba van: **fordítási hiba** (a program el sem indul, a Hibalista mutatja) és **futásidejű hiba** (elindul, de menet közben összeomlik). A „véletlenszerű összeomlások” szinte mind az utóbbiak.

ÍRATLAN SZABÁLYOK

- 1 Olvasd el a **Hibalistát** (Error List), és a **legfelső** hibát javítsd először: a többi gyakran csak következménye.
- 2 A **sárga figyelmeztetést** se hagyd halmozódni: gyakran valódi hibára utal (pl. null).
- 3 Ments és építs **gyakran** (Ctrl+S, Ctrl+Shift+B): kis lépésekben könnyebb hibát találni.
- 4 Újrafuttatás előtt **állítsd le** az előző példányt: a futó .exe zárolja a fájlt, és a build hibázhat.

```
projekt.sln          <- megoldás (csak egy "tartalomjegyzék")
└─ projekt/
    ├── projekt.csproj <- a projekt beállításai (.NET verzió)
    ├── Program.cs     <- ITT írod a kódot
    ├── bin/Debug/...   <- az elkészült program (.exe)
    └─ obj/             <- ideiglenes fájlok
```

- A **.sln** (solution = megoldás) nem tartalmaz kódot, csak azt tudja, mely projektek tartoznak össze. Duplakattal ezt nyisd meg.
- A **.csproj** a projekt „személyije”: milyen .NET verziót használ, milyen beállításokkal.
- A **Program.cs** a fő kódfájl. Az **F5** a **bin/Debug** mappába építi a programot. Ide kerülnek a relatív útvonalú fájlok is (lásd: fájlkezelés).

RÉSZLETES MAGYARÁZAT

- A **solution** (.sln) több projektet is összefoghat; a **projekt** (.csproj) egy lefordítható egység (itt: egy konzolprogram).
- A fordító a **.cs** fájlokból **.dll/.exe** fájlt készít a **bin** mappába. Az **obj** a fordítás köztes termékeit tartalmazza.
- A **bin\Debug** és **bin\Release** külön mappa: a Debug hibakereshető, a Release gyorsabb és kiadásra való.
- Relatív útvonalú fájl ("**adat.txt**") a **futó program mappájában** keletkezik, vagyis a **bin\Debug\...** alatt, nem a **Program.cs** mellett.

ÍRATLAN SZABÁLYOK

- 1 A **bin** és **obj** mappát **ne szerkeszd kézzel**, és ne tárolj bennük fontos fájlt: a „Clean/Rebuild” törölheti.
- 2 A projektmappát **ne nevezd át** a Visual Studión kívül, mert a **.sln** útvonala elromlik.
- 3 Megosztáskor az **egész projektmappát** add át (a **bin** és **obj** elhagyható), ne csak a **Program.cs**-t.
- 4 Egy projektben csak **egy Main** lehet, különben „több belépési pont” hibát kapsz.

```
using System;

namespace projeckt
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Helló, világ!");
        }
    }
}
```

- **using** – eszköztár behúzása (itt: a Console miatt)
- **namespace** – a kódod „mappája”, a projekt neve
- **class Program** – az osztály, amiben a program él
- **Main** – **innen indul** a program, itt kezdődik a futás
- Minden utasítás végére **;** kell, a **{ }** pedig összetartozó blokkot jelöl

Újabb .NET-ben (6+) a sablon rövidebb is lehet: elég egy sor, `Console.WriteLine("Helló!");`. Ugyanazt csinálja.

RÉSZLETES MAGYARÁZAT

- A program **fentről lefelé, sorról sorra** fut a `Main` első sorától az utolsóig. Ha a `Main` véget ér, a program is.
- A `{ }` **blokkot** határol; minden nyitó zárójelnek kell záró párja. A behúzás nem kötelező, de nélküle nem látod a szerkezetet (Ctrl+K, Ctrl+D rendez).
A `{ }` blokkot határol; minden nyitó zárójelnek kell záró párja. A behúzás nem kötelező, de nélküle nem látod a szerkezetet (Ctrl+K, Ctrl+D rendez).
- A C# **kis- és nagybetű érzékeny**: `Console` ≠ `console`, `Main` ≠ `main`.
- A szöveg **idézőjelbe** ("`...`"), egyetlen karakter **aposztrófba** ('`a`') kerül. A `string[] args` a parancssori paraméter; most nem használjuk.

ÍRATLAN SZABÁLYOK

- 1 Írd le **egyszerre** a `{` és a `}` jelet, és csak utána tölts ki a belsejét: így nem marad lezáratlan blokk.
- 2 Utasítás végén `;` kell, blokk (`if`, `for`, `class`) végén **nem**.
- 3 Ne duplikáld a `Main`-t, és ne tegyél utasítást a metóduson **kívülre**: fordítási hiba.
- 4 Ha a konzolban hibás az ékezet, állítsd be:
`Console.OutputEncoding = System.Text.Encoding.UTF8;`

```
Console.WriteLine("Ez új sorba ír");  
Console.Write("Ez ugyanabba a sorba: ");  
  
Console.Write("Mi a neved? ");  
string nev = Console.ReadLine() ?? "";  
  
Console.WriteLine("Szia, " + nev + "!");  
Console.WriteLine($"Szia, {nev}!"); // ugyanaz, szebben
```

- `WriteLine` kiír és sort emel, a `Write` nem.
- `ReadLine` beolvassa, amit a felhasználó begépel, **mindig szöveggént**.
- A `$"...{nev}..."` az **interpolált szöveg**: a `{ }` közé változót írhatsz.

A `?? ""` csak a sárga figyelmeztetést (CS8600) kerüli el arra az esetre, ha a beolvasás üres lenne. Elhagyhatod, nem hiba.

RÉSZLETES MAGYARÁZAT

- A konzol **szöveges** be- és kimenet. A `ReadLine()` addig vár, amíg a felhasználó Entert nyom, és az **egész sort szöveggént** adja vissza.
- Ha a bemenet le van zárva (pl. fájlból irányított), a `ReadLine()` **null**-t ad: ezért a típusa `string?` („lehet null”).
- A `$"..."` interpolált szövegben a `{ }` közötti **kifejezés kiértékelődik**: `{a + b}`; formázás: `{ar:F2}` (két tizedes).
- Ha magát a kapcsos zárójelet akarod kiírni a `$"..."` szövegben, kettőzd: `{{` és `}}`.

ÍRATLAN SZABÁLYOK

- 1 Bekérés előtt **írd ki, mit vársz** (`Console.Write("Kor: ")`), különben a felhasználó nem tudja, mit kell tennie.
- 2 A `ReadLine` eredményét **soha ne tekintsd számnak**: konvertáld és ellenőrizd (lásd: konverzió).
- 3 A visszatérési értéken ne hívj azonnal metódust (`.Trim()`), ha lehet null: használj `?? ""`-t.
- 4 A konzol a program végén bezárul: futtass **Ctrl+F5**-tel, vagy írd a végére `Console.ReadKey();`-t.

```
int kor = 17;           // egész szám
double magassag = 1.82; // tizedes tört
string nev = "Anna";    // szöveg
char betu = 'A';        // egyetlen karakter
bool diak = true;       // igaz / hamis
var szam = 10;           // a C# kitalálja: int
const double PI = 3.14;  // állandó, nem változtatható

kor = 18;               // új érték
kor = kor + 1;          // most 19
kor++;                  // most 20
```

- A változó = **név + típus + érték**.
- Egy változót **egyszer** deklarálsz (típussal), utána csak értéket adsz neki.
- Névadás: kisbetűvel kezd, ne kezdődjön számmal, ne legyen benne szóköz. Pl. **maxPont**.
- A tizedes **a kódban mindig pont**: **1.82**.

RÉSZLETES MAGYARÁZAT

- A változó a **memória egy darabja**, amihez nevet és típust adsz. A típus mondja meg, mekkora helyet foglal és mit lehet vele csinálni.
- Az **= értékadás**, nem egyenlőség! A jobb oldalt kiértékeli, az eredményt a bal oldali változóba teszi: `kor = kor + 1`.
- A változót **használat előtt értékkel kell ellátni**, a fordító nem engedi a kezdőérték nélkül olvasni.
- A változó csak azon a **blokkon belül él**, ahol deklaráltad (hatókör): egy `{ }` közt létrehozott változó kívülről nem látszik.
- A `var` nem „bármilyen típus”: a fordító a jobb oldalból **kitalálja** a típust, és az utána nem változik.

ÍRATLAN SZABÁLYOK

- 1 Adj **beszédes nevet**: `maxPont`, nem `x1`.
Változó: kisbetűvel (`kor`), osztály/metódus: nagybetűvel (`Car`, `Mentes`).
- 2 Ne használd ugyanazt a nevet két **egymásba ágyazott** hatókörben: fordítási hiba.
- 3 Az `int` kb. $\pm 2,1$ milliárdig tart: előlött **csendben túlcsoordul** és rossz értéket ad. Nagy számhoz `long`.
- 4 Értéket **mindig adj** a változónak használat előtt (`int osszeg = 0;`): ne bízz a „majd lesz valami”-ben.

Típus	Mit tárol?	Példa
<code>int</code>	egész szám	42, -7
<code>double</code>	tizedes tört	3.14
<code>decimal</code>	pontos tizedes (pénz)	19.99m
<code>string</code>	szöveg	"Anna"
<code>char</code>	egyetlen karakter	'A'
<code>bool</code>	igaz vagy hamis	true, false

Szabály: a típus határozza meg, mit tehetsz az értékkel. `int`-ből nem lesz szöveg magától, és `string`-gel nem tudsz számolni. Erre jó a konverzió (következő dia).

RÉSZLETES MAGYARÁZAT

- A típus két dolgot dönt el: **mennyi memória** kell, és **milyen műveletek** lehetségesek. **int**: 4 bájt ($\pm 2,1$ milliárd), **long**: 8 bájt, **double**: 8 bájt (~15 értékes jegy), **decimal**: 16 bájt, pontos tízes alapú tört.
- Az **int** és **double** **értéktípus** (a változóban maga az érték van), a **string** **referenciatípus** (hivatkozás), de a szöveg **módosíthatatlan**: minden „módosítás” új szöveget ad.
- A **double** kettes számrendszerben tárol, ezért **0.1 + 0.2** nem pontosan **0.3**. A **decimal** pénzhez való.
- A **null** jelentése „nincs érték”. A **string** lehet null, az **int** nem (kivéve **int?**).

ÍRATLAN SZABÁLYOK

- 1 Pénzhez **decimal** (**19.99m**), soha **double**.
- 2 Két **double**-t ne hasonlíts **==**-vel: használd **Math.Abs(a - b) < 0.0001**-et.
- 3 Amivel nem számolsz (telefonszám, irányítószám), tárold **string**-ként: az elején lévő 0 nem vesz el.
- 4 Szöveg metódusa előtt gondolj rá: **lehet null** → **NullReferenceException**. Ellenőrizd vagy használd a **?.** operátort.

```
Console.Write("Hány éves vagy? ");
string szoveg = Console.ReadLine() ?? "";

// 1) Egyszerű, de elszáll, ha nem szám:
int kor = int.Parse(szoveg);

// 2) Biztonságos:
if (int.TryParse(szoveg, out int jo))
    Console.WriteLine("Jövőre " + (jo + 1) + " leszel.");
else
    Console.WriteLine("Ez nem szám!");

double d = 3.99;
int egesz = (int)d;           // 3 (levágja a tizedest)
string s = 42.ToString();    // "42"
```

- A `ReadLine` mindig **szöveget** ad, ezt kell számmá alakítani.
- `int.Parse` hibás bemenetnél **kivételt dob** (a program összeomlik).
- `int.TryParse` nem omlik össze: igaz/hamis értéket ad vissza.

Magyar gépen: `double.Parse` beolvasásnál a gép nyelvi beállítása számít. Ott a tizedesjel a **vessző** (1,5), a pont (1.5) hibát okozhat.

RÉSZLETES MAGYARÁZAT

- Kétféle konverzió van: **implicit** (automatikus, nincs adatvesztés: `int` → `double`) és **explicit** (kényszerített, adat veszhet: `(int)3.99` → 3).
- A `Parse` azt mondja: „szerintem ez szám”. Ha nem az, **kivételt dob** (`FormatException`). A `TryParse` megpróbálja, `bool`-t ad vissza, az eredmény az `out` paraméterben van.
- Ha a `TryParse` hamisat ad, az `out` változó `0` (alapérték): úgy nem szabad használni, mintha sikerült volna.
- A `Convert.ToInt32` is létezik: `null`-ra 0-t, hibás szövegre kivételt ad.

ÍRATLAN SZABÁLYOK

- 1 **Felhasználói bevitelre mindig `TryParse`.**
`Parse`-t csak megbízható adatnál, `try/catch`-cel.
- 2 A `TryParse` visszatérési értékét ellenőrizd, ne csak az `out` változót használd.
- 3 Túl nagy számnál `OverflowException` jön (`int.Parse("3000000000")`): `long` vagy `TryParse`.
- 4 Magyar gépen a tizedesjel a vessző: ha pontot kell fogadnod, használj `CultureInfo.InvariantCulture`-t (`using System.Globalization;`).

Fajta	Operátorok	Példa
Számolás	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code>	<code>7 % 2 → 1</code> (maradék)
Összehasonlítás	<code>==</code> <code>!=</code> <code><</code> <code>></code> <code><=</code> <code>>=</code>	<code>kor >= 18</code>
Logikai	<code>&&</code> (és) <code> </code> (vagy) <code>!</code> (nem)	<code>kor >= 18 && diak</code>
Értékadás	<code>=</code> <code>+=</code> <code>-=</code> <code>++</code> <code>--</code>	<code>pont += 5;</code>

Gyakori csapda: egész számok osztása egészet ad. `7 / 2` az `3`, nem `3,5`! Ha tizedes eredmény kell, írd így: `7 / 2.0`. Az **egyenlőség vizsgálata** `==`, az `=` értéket ad.

RÉSZLETES MAGYARÁZAT

- Az operátoroknak **erőssége** (precedenciája) van: `*`, `/`, `%` előbb számolódik, mint `+`, `-`. Zárójellel bármikor felülírod: `(a + b) * c`.
- **Egész osztás**: ha mindkét szám egész, az eredmény is egész (a tört levágódik). A `%` a maradék: `10 % 3 = 1`; páros, ha `n % 2 == 0`.
- Az `&&` és `||` **rövidzáras**: ha a bal oldal már eldönti az eredményt, a jobb oldalt **ki sem értékeli**. Ez védelem is: `s != null && s.Length > 0`.
- `i++` önálló sorban ugyanaz, mint `i = i + 1`.

ÍRATLAN SZABÁLYOK

- 1 **Osztás előtt** ellenőrizd az osztót: egész osztásnál `DivideByZeroException`, `double`-nál `Infinity`.
- 2 `==` összehasonlít, `=` értéket ad. `if (x = 5)` hibát jelez, de `bool` változónál **észrevétlenül** elronthatod.
- 3 Az `&&` bal oldalára a **védő feltétel** kerüljön (null, tartomány), jobbra az, ami összeomolhat.
- 4 Bonyolult kifejezést **bontsd részekre**, és adj nevet a köztes eredménynek.

```
string s = "Hello Világ";

Console.WriteLine(s.Length);           // 11
Console.WriteLine(s.ToUpper());        // HELLO VILÁG
Console.WriteLine(s.ToLower());        // hello világ
Console.WriteLine(s.Contains("Vil")); // True
Console.WriteLine(s.Substring(0, 5));  // Hello
Console.WriteLine(s.Replace("l", "L"));
Console.WriteLine(s.Trim());           // szélső szóközök le

string[] szavak = "alma,körte,szilva".Split(',');
Console.WriteLine(szavak[1]);          // körte
```

- A szövegen sok **beépített művelet** van, ponttal hívod őket.
- A karakterek számozása **0-tól** indul.
- A **Split** szétvágja a szöveget, ez a fájlkezelésnél nagyon jól jön majd.

RÉSZLETES MAGYARÁZAT

- A `string` **karakterek sorozata**, indexelhető: `s[0]` az első, az utolsó `s[s.Length - 1]`. Az index **0-tól** indul.
- A szöveg **módosíthatatlan** (immutable): a `s.ToUpper()` új szöveget ad vissza, `s` nem változik. Ezért kell: `s = s.ToUpper();`.
- Az `==` jó összehasonlításra, de kis-nagybetű számít: `"a" == "A"` hamis. Kis-nagybetűtől függetlenül: `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)`.
- Sok összefűzés ciklusban lassú: használj `StringBuilder`-t.

ÍRATLAN SZABÁLYOK

- 1 A `Substring(start, hossz)` kivételt dob, ha túllépsz a végén: előbb nézd meg a `Length`-et.
- 2 `s[5]` előtt győződj meg róla, hogy `s.Length > 5`.
- 3 A művelet eredményét **mentsd el** (`s = s.Trim();`), különben nincs hatása.
- 4 Üresség vizsgálatához: `string.IsNullOrEmpty(s)` (null, üres és csak szóközös szöveget is elkap).

```
int pont = 78;

if (pont >= 90)
    Console.WriteLine("Jeles");
else if (pont >= 75)
    Console.WriteLine("Jó");
else if (pont >= 50)
    Console.WriteLine("Elégséges");
else
    Console.WriteLine("Elégtelen");

int kor = 20;
bool diak = true;
if (kor >= 18 && diak)
{
    Console.WriteLine("Felnőtt diák");
    Console.WriteLine("Több sor = kell a { }");
}
```

- A program **fentről lefelé** vizsgálja a feltételeket, és az **első igaz** ágat hajtja végre.
- Egy utasításnál elhagyható a **{ }**, többnél **kötelező**.
- A feltétel **bool** (igaz/hamis) értékű kifejezés legyen.

RÉSZLETES MAGYARÁZAT

- Az `if` kifejezése `bool` (igaz/hamis) legyen. A C# nem fogadja el a „0 = hamis” szabályt, mint a C.
- A feltételeket **fentről lefelé** értékeli ki, az **első igaz ágat** hajtja végre, a többi átugorja. Ezért a **sorrend számít**: előre a szűkebb feltétel (`pont >= 90`).
- Az `else` mindig a hozzá **legközelebbi if**-hez tartozik; `{ }` nélkül ez félreérthető.
- Összetett feltételt tehetsz egy `bool` változóba: `bool felnott = kor >= 18;`, így olvashatóbb.

ÍRATLAN SZABÁLYOK

- 1 **Mindig használj `{ }`-t**, akkor is, ha egy sor van: a később beírt második sor csendben kilóg a feltételből.
- 2 Ne tegyél `;`-t az `if ( ... )` után: üres törzset jelent, a következő blokk **mindig** lefut.
- 3 Gondolj a **szélső esetekre**: a határérték (`>=` vagy `>?`), 0, negatív szám, üres szöveg.
- 4 Ne ismételd ugyanazt a feltételt: tedd változóba vagy metódusba (`Felnott(kor)`).

```
Console.Write("A hét napja (1-7): ");
int nap = int.Parse(Console.ReadLine() ?? "1");

switch (nap)
{
    case 1:
        Console.WriteLine("Hétfő");
        break;
    case 2:
        Console.WriteLine("Kedd");
        break;
    case 6:
    case 7:
        Console.WriteLine("Hétfő");
        break;
    default:
        Console.WriteLine("Hétköznapi");
        break;
}
```

- A **switch** akkor jó, ha **egy érték** sok lehetséges esetét kell szétválogatni.
- Minden **case** végén kell a **break**.
- A **default** az „egyik sem” ág.
- Több **case** egymás alatt = ugyanaz az ág.

RÉSZLETES MAGYARÁZAT

- A `switch` egy értéket **több lehetséges értékkel** hasonlít össze. Hosszú `else if` lánc helyett olvashatóbb.
- C#-ban **nincs „átcsorgás”**: minden nem üres `case`-t le kell zárni (`break`, `return`, `throw`). Az üres, egymás alatti `case`-ek viszont **összevonódnak** (6 és 7).
- A `default` ág akkor fut, ha egyik `case` sem talált: a váratlan érték se maradjon kezeletlen.
- Újabb C#-ban van `switch` kifejezés is: `var nev = nap switch { 1 => "Hétfő", _ => "Más" };`

ÍRATLAN SZABÁLYOK

- 1 **Mindig írd `default` ágat**, és jelezd benne a váratlan értéket (üzenet, napló).
- 2 A `case` értéke **állandó** legyen (szám, szöveg, `enum`), változó nem lehet.
- 3 Szövegnél normalizálj előtte: `.Trim().ToLower()`, hogy az `"Igen "` és `"igen"` ugyanaz legyen.
- 4 Ha több ágnak ugyanaz a dolga, rakd a `case`-eket egymás alá, ne másold a kódot.

Ciklusok: mire jók, és melyiket mikor használjuk?

A **ciklus** (angolul: loop) egy kódrészt **többször egymás után** végrehajt. Képzeld el, hogy 100-szor kell leírnod egy mondatot: nem írod le százszor, hanem azt mondod: „ismételd 100-szor”. A ciklus pontosan ezt csinálja.

Ciklus	Mikor használd?	Mikor ellenőrzi a feltételt?	Legalább hányszor fut?
<code>for</code>	Előre tudod, hányszor kell ismételni (pl. 10-szer)	Minden kör előtt	0-szor
<code>while</code>	Nem tudod, hányszor , addig megy, amíg egy feltétel igaz	Minden kör előtt	0-szor
<code>do-while</code>	Legalább egyszer biztosan le kell futnia (pl. menü, bekérés)	Minden kör után	1-szer
<code>foreach</code>	Egy lista vagy tömb minden elemén végig kell menni	nincs feltétel	0-szor (üres listánál)

Hogyan válassz? Tudod a darabszámot → `for`. Nem tudod, de van „amíg” feltétel → `while`. Előbb csinálni kell valamit, csak utána derül ki, kell-e még → `do-while`. Gyűjtemény elemei → `foreach`.

RÉSZLETES MAGYARÁZAT

- Minden ciklusnak három eleme van: **kezdőállapot**, **feltétel**, **léptetés** (az állapot megváltoztatása). Ha ezek bármelyike hiányzik vagy hibás, a ciklus hibázik.
- A ciklus **törzse** a `{ }` közötti rész, ezt ismétli. Egy ismétlés neve: **iteráció** (kör).
- A feltétel mindig **bool**. Az ismétlések számát vagy előre tudod (**for**), vagy egy állapot dönti el (**while**).
- Bármelyik ciklus átírható másikba: a választás az **olvashatóságról** szól, nem a lehetőségről.

ÍRATLAN SZABÁLYOK

- 1 Ciklus írása előtt kérdezd meg: **mikor áll meg?** Ha nem tudod megmondani, végtelen ciklus lesz.
- 2 Gondold végig a **0 és az 1 kör** esetét: üres lista, azonnal hamis feltétel.
- 3 A **for** ciklusváltozóját **ne módosítsd** a törzsben (kivéve a léptetést): követhetetlen lesz.
- 4 Ha nem érted, mit csinál a ciklus, tegyél bele **Console.WriteLine**-t vagy **töréspontot** (F9).

for (**int i = 1;** **i <= 5;** **i++**)

1) Kezdőérték

Egyszer fut le, a ciklus elején. Itt hozod létre a számlálót: `int i = 1` (az `i` az első kör értéke).

2) Feltétel

Minden kör előtt ellenőrzi. Ha igaz, jöhet a kör, ha hamis, a ciklus véget ér.

3) Léptetés

Minden kör végén lefut. `i++` = növekszt egyet (ugyanaz, mint `i = i + 1`).

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine("Kör: " + i);    // ciklustörzs
}
Console.WriteLine("Vége!");          // a ciklus után
```

A `{ }` közötti **ciklustörzs** ismétlődik. Kimenet:
`Kör: 1 ... Kör: 5`, majd `Vége!`.

A sorrend: **1) egyszer** → **2) feltétel** → **törzs** → **3) léptetés** → **2) feltétel** → **törzs** → **3)** ... egészen addig, amíg a 2) hamis nem lesz.

Hogyan fut le a for lépésről lépésre?

Kövessük végig ezt: `for (int i = 1; i <= 5; i++) Console.WriteLine(i);`

Lépés	i értéke	i <= 5 ?	Mi történik?
Indulás	1	–	A kezdőérték beállítása (csak egyszer)
1. kör	1	igaz	Kiírja: 1 , utána <code>i++</code> → <code>i</code> lesz 2
2. kör	2	igaz	Kiírja: 2 , utána <code>i</code> lesz 3
3. kör	3	igaz	Kiírja: 3 , utána <code>i</code> lesz 4
4. kör	4	igaz	Kiírja: 4 , utána <code>i</code> lesz 5
5. kör	5	igaz	Kiírja: 5 , utána <code>i</code> lesz 6
Vége	6	hamis	A ciklus véget ér, a program a ciklus utáni sorral folytatódik

Tipp kezdőknek: ha nem érted, mit csinál egy ciklus, rajzolj ilyen táblázatot papíron. Ezt hívják **asztali tesztnek**, és programozók is használják.

```
// 1) Visszafelé számolás
for (int i = 10; i >= 1; i--)
    Console.WriteLine(i);

// 2) Kettesével lépés (páros számok)
for (int i = 0; i <= 20; i += 2)
    Console.WriteLine(i);

// 3) Összegzés: 1 + 2 + ... + 100
int osszeg = 0;
for (int i = 1; i <= 100; i++)
{
    osszeg += i;          // osszeg = osszeg + i
}
Console.WriteLine(osszeg); // 5050
```

- A **léptetés** nem csak `i++` lehet: `i--` (csökken), `i += 2` (kettesével).
- **Összegzés**: kell egy változó a ciklus **előtt** (`osszeg = 0`), amit minden körben növelsz.
- A számlálót (`i`) a ciklus **belsejében** használhatod, de a cikluson **kívül már nem létezik**.
- Tömbnél: `for (int i = 0; i < tomb.Length; i++)`, mert az indexek **0-tól** indulnak.

Mini-feladat: Írd ki a 3 többszöröseit 3-tól 30-ig!

RÉSZLETES MAGYARÁZAT

- A **for** három része három helyen fut: **kezdőérték** (1×), **feltétel** (minden kör előtt), **léptetés** (minden kör végén). A `for (int i = 0; i < n; i++)` pontosan `n` kört ad: `0, 1, ..., n-1`.
- A számláló (`i`) **csak a ciklusban él**. Ha a cikluson kívül is kell az értéke, a ciklus előtt deklaráld.
- Tömbnél a feltétel `i < tomb.Length`, nem `<=`, mert az utolsó index `Length - 1`.
- A feltétel **minden körben újra kiértékelődik**: ha `lista.Count` szerepel benne, és közben törölsz, a körök száma is változik.

ÍRATLAN SZABÁLYOK

- 1 Tömbnél `<`, számtartománynál (1–N) gyakran `<=`. **Írd fel papíron** az első és utolsó értéket.
- 2 Ne töröld és ne adj hozzá elemet a listához, miközben előre járod be: menj **hátról** (`i--`), vagy másolaton dolgozz.
- 3 A `for (...)` sor végére **soha ne tegyél `;`-t**: üres ciklus lesz.
- 4 Az összegző változót (`osszeg = 0`) a ciklus **előtt** nullázd, ne benne.
- 5 A számláló legyen `int`: `double` lépésnél (0.1) a hibák halmozódnak.

A while ciklus: „amíg igaz, ismételd”

```
// 1) Visszaszámlálás
int n = 3;
while (n > 0)
{
    Console.WriteLine(n);
    n--;           // enélkül végtelen ciklus lenne!
}
Console.WriteLine("Indítás!");

// 2) Addig kérünk számot, amíg nem pozitív
int szam = 0;
while (szam <= 0)
{
    Console.Write("Pozitív szám: ");
    int.TryParse(Console.ReadLine(), out szam);
}
Console.WriteLine("Rendben: " + szam);
```

- Felépítés: `while (feltétel) { törzs }`
- A feltételt **a kör előtt** ellenőrzi. Ha már az elején hamis, a törzs **egyszer sem** fut le.
- A törzsben **változtatni kell valamit**, ami egyszer hamissá teszi a feltételt (itt: `n--`, vagy a felhasználó új értéket ad).
- Ha a feltétel soha nem lesz hamis, **végtelen ciklus** keletkezik (leállítás: Ctrl+C a konzolban, vagy a Stop gomb).

Akkor használd, ha **nem tudod előre**, hányszor kell ismételni, pl. „amíg a felhasználó nem ír helyes értéket”.

RÉSZLETES MAGYARÁZAT

- A **while (feltétel)** minden kör előtt kiértékeli a feltételt: ha igaz, lefut a törzs, aztán újra ellenőriz. Hamis feltételnél a ciklus utáni sorral folytatódik.
- A törzsnek **közelítenie kell a megálláshoz**: változtatnia kell valamin, ami a feltételben szerepel (számláló, bemenet, állapot).
- Tipikus minták: **bemenet-ellenőrzés** (addig kérd, amíg jó), **keresés** (amíg meg nem találod), **szimuláció** (amíg van hátra).
- A **while (true)** szándékosan végtelen: **break**-kel lépsz ki. Ez tudatos döntés legyen, ne baleset.

ÍRATLAN SZABÁLYOK

- 1 A feltételben szereplő változót a törzsben **frissítsd**, és a frissítés ne kerüljön olyan **if** mögé, ami néha nem fut le.
- 2 Bekérésnél adj **kiutat** (0 = vége, vagy max. 3 próba), ne zárd a felhasználót végtelen körbe.
- 3 Fájl- és hálózati olvasásnál is legyen megállási feltétel (**EndOfStream**).
- 4 Ha a program „lefagy”, gyanakodj **végtelen ciklusra**: állítsd le (Stop / Ctrl+C), és nézd meg a feltételt.

A do-while ciklus: „előbb csináld, aztán kérdezz”

```
int valasz;

do
{
    Console.Write("Írj egy számot 1 és 10 között: ");
    int.TryParse(Console.ReadLine(), out valasz);
}
while (valasz < 1 || valasz > 10);    // <- itt pontosvessző!

Console.WriteLine("Elfogadva: " + valasz);

// Egyszerű menü:
string menu;
do
{
    Console.WriteLine("1 - Köszönés | 0 - Kilépés");
    menu = Console.ReadLine() ?? "";
    if (menu == "1") Console.WriteLine("Szia!");
}
while (menu != "0");
```

- Felépítés: `do { törzs } while (feltétel);`. A `while` után **pontosvessző** kell!
- A feltételt **a kör után** ellenőrzi, ezért a törzs **mindig legalább egyszer** lefut.
- Tökéletes **bekéréshez** és **menühöz**: előbb meg kell kérdezni valamit, csak utána derül ki, jó-e a válasz.
- A bekérős példa így rövidebb, mint a `while`-os, mert nem kell előre kitalálni egy „hamis kezdőértéket”.

RÉSZLETES MAGYARÁZAT

- A `do { } while (...);` a törzset **először lefuttatja**, és csak utána értékeli a feltételt. Így **legalább egyszer** lefut.
- Tipikus használat: menü, bekérés, „újra?” kérdés: olyasmi, amit **előbb meg kell csinálni**, hogy a feltételt egyáltalán ki lehessen értékelni.
- A feltételben szereplő változót a **do előtt** kell deklarálni: a blokkon belül létrehozott változó a `while (...)` sorban már nem látszik.
- A záró `;` a `while (...)` után **kötelező**.

ÍRATLAN SZABÁLYOK

- 1 Ha a változó a törzsben kap értéket, deklaráld előtte: `int valasz;`.
- 2 Ne felejtsd el a záró **pontosvesszőt**.
- 3 A feltétel az „ismételj, ha **rossz** a válasz”: `while (v < 1 || v > 10)`. Ellenőrizd a **De Morgan**-szabállyal: `!(a && b) = !a || !b`.
- 4 Menünél mindig legyen **kilépés** opció, különben csak a Ctrl+C állítja le.

while vagy do-while? A különbség

```
int x = 10;

// while: a feltétel előbb - 10 < 5 hamis, EGYSZER SEM fut
while (x < 5)
{
    Console.WriteLine("while törzs");
}

// do-while: a törzs előbb - EGYSZER lefut
do
{
    Console.WriteLine("do-while törzs");
}
while (x < 5);

// Kimenet:
// do-while törzs
```

- Mindkettőnél a feltétel $x < 5$, ami **kezdetben hamis** (mert $x = 10$).
- A **while** hamis feltételnél egyetlen kört sem csinál.
- A **do-while** először lefuttatja a törzset, és csak **utána** vizsgál. Ezért **pontosan egyszer** fut le.

Egyszerű emlékeztető: **while** = „előbb gondolkodj, aztán csináld”. **do-while** = „előbb csináld, aztán gondolkodj”.

A foreach ciklus: minden elemen végig

```
string[] nevek = { "Anna", "Bence", "Cili" };

// foreach: egyszerű, nincs index
foreach (string nev in nevek)
{
    Console.WriteLine("Szia, " + nev + "!");
}

// ugyanez for-ra: itt van index
for (int i = 0; i < nevek.Length; i++)
{
    Console.WriteLine((i + 1) + ". " + nevek[i]);
}

List<int> pontok = new List<int> { 10, 20, 30 };
int ossz = 0;
foreach (int p in pontok)
    ossz += p;
Console.WriteLine(ossz);    // 60
```

- Felépítés: **foreach** (típus név in gyűjtemény). A **nev** minden körben a **következő elem**.
- Nem kell számlálni, nem lehet elszámolni, ezért **biztonságosabb** a **for**-nál.
- A **foreach** közben **nem módosíthatod** a gyűjteményt (nem adhatsz hozzá, és nem vehetsz el elemet).
- Ha kell az **index** (sorszám), akkor használj **for**-t.

RÉSZLETES MAGYARÁZAT

- A **foreach** az elemeket **egyenként, sorrendben** adja. A ciklusváltozó **csak olvasható**: a `nev = "x"` fordítási hiba.
- Mögötte az **IEnumerable** felület áll: bármi bejárható vele (tömb, **List**, **string**, **Dictionary**, fájl sorok).
- Nincs index, így nem lehet **eggyel elcsúszni** vagy a tömbből kilógni: ezért a legbiztonságosabb ciklus.
- Ha az elem **objektum**, annak tulajdonságait módosíthatod (`a.Color = "Red"`), mert hivatkozást kapsz. Csak a változó cseréje tilos.

ÍRATLAN SZABÁLYOK

- 1 Ne módosítsd a gyűjteményt (**Add/Remove**) a **foreach** közben:
InvalidOperationException.
Alternatíva: **for** hátulról, vagy **RemoveAll**.
- 2 Ha **sorszám** is kell, használj **for**-t.
- 3 A gyűjtemény **ne legyen null**: **foreach** (`var x in null`) → **NullReferenceException**.
- 4 A ciklusváltozó neve legyen egyes számú (**auto**), a gyűjteményé többes (**autok**): olvasható marad.

```
for (int i = 1; i <= 10; i++)
{
    if (i == 3)
        continue;           // ugrás a KÖVETKEZŐ körre

    if (i == 7)
        break;               // KILÉPÉS a ciklusból

    Console.WriteLine(i);
}
// Kimenet: 1 2 4 5 6

// Tipikus használat: keresés
int[] szamok = { 4, 9, 15, 22, 8 };
foreach (int sz in szamok)
{
    if (sz > 20)
    {
        Console.WriteLine("Találtam: " + sz);
        break;               // nincs értelme tovább keresni
    }
}
```

- **break**: azonnal **kilép** a legközelebbi ciklusból, és a ciklus utáni sorral folytatja.
- **continue**: **kihagyja** a kör hátralévő részét, és a következő körrel folytatja.
- Nálunk a 3-ast a **continue** kihagyta, a 7-esnél a **break** leállította a ciklust, ezért 7 és utána semmi nem íródott ki.
- A **break** ugyanúgy működik **while**, **do-while**, **foreach** és **switch** esetén is.

RÉSZLETES MAGYARÁZAT

- A **break** a **legközelebbi** (legbelső) ciklust vagy **switch**-et hagyja el. A **continue** csak a **kör hátralévő részét** hagyja ki, és a következő körre ugrik (**for**-nál a léptetés még lefut).
- **while**-ban a **continue** **veszélyes**: ha a léptetés a törzs végén van, a **continue** átugorja, és végtelen ciklus lesz.
- Egymásba ágyazott ciklusnál a **break** csak a belsőt állítja le. Mindkettőből kilépni: **bool** jelző, vagy tedd metódusba és használj **return**-t.
- A **korai kilépés** (guard) kevesebb egymásba ágyazott **if**-et jelent: olvashatóbb.

ÍRATLAN SZABÁLYOK

- 1 **while**-ban **continue** előtt **gondoskodj a léptetésről (n++)**.
- 2 Mértékkel használd: ha egy ciklusban 3 **break/continue** van, **bontsd metódusokra**.
- 3 Keresésnél jegyezd fel (**bool talalt**), hogy találtál-e: a ciklus után nem tudod, miért lépett ki.
- 4 A **goto**-t kerüld: követhetetlenné teszi a kódot.

```
// Szorzótábla 5x5
for (int sor = 1; sor <= 5; sor++)
{
    for (int oszlop = 1; oszlop <= 5; oszlop++)
    {
        Console.Write(sor * oszlop + "\t");
    }
    Console.WriteLine();    // sor vége: új sor
}
```

```
// Kimenet:
// 1  2  3  4  5
// 2  4  6  8  10
// 3  6  9  12 15
// 4  8  12 16 20
// 5  10 15 20 25
```

- Egy ciklus törzsében **másik ciklus** is lehet: a **külső** egyszer lép, a **belső** közben teljesen lefut.
- Itt a külső ciklus a **sorokat**, a belső az **oszlopokat** adja. 5 sor × 5 oszlop = **25 kör**.
- A **\t** tabulátor (szép oszlopokhoz), a **Write** ugyanabba a sorba ír, a **WriteLine()** új sort kezd.
- Használd különböző nevű számlálókat (**sor**, **oszlop**), hogy ne keveredjen össze.

Mini-feladat: rajzolj * karakterekből háromszöget (1, 2, 3, 4, 5 csillag soronként)!

RÉSZLETES MAGYARÁZAT

- A belső ciklus **minden külső körben újraindul**, és teljesen lefut. Az összes kör száma: külső $\times$ belső ($5 \times 5 = 25$).
- A külső ciklus általában a **sorokat**, a belső az **oszlopokat** adja (táblázat, mátrix, rajz).
- A belső ciklus kezdőértéke függhet a külsőtől (**$j = i$**): háromszög alakú bejárás.
- A futásidő **négyzetesen nő**: $1000 \times 1000 =$ egymillió kör.

ÍRATLAN SZABÁLYOK

- 1 Használj **különböző neveket** (**sor**, **oszlop**): ugyanazt az **i**-t kétszer deklarálni hiba.
- 2 A sortörés (**Console.WriteLine()**) a **belső ciklus után**, a külsőben legyen.
- 3 Három-négy szintnél mélyebb egymásba ágyazás: **bontsd metódusokra**.
- 4 Nagy adatnál gondold végig, hány kört csinál: a kétszeres ciklus ezrekre már lassú lehet.

Hiba	Mi történik?	Hogyan javítsd?
Végtelen ciklus	A feltétel soha nem lesz hamis (pl. elfelejtetted az <code>n--</code> -t). A program lefagyni látszik.	Ellenőrizd, hogy a törzs változtat-e valamit. Megállítás: Ctrl+C
Eggyel elcsúszás (off-by-one)	<code>i < 5</code> és <code>i <= 5</code> nem ugyanaz: eggyel kevesebb vagy több kör lesz.	Rajzold fel az asztali tesztet az első és az utolsó körre
Index kicsúszása	<code>for (i = 0; i <= tomb.Length; ...)</code> → <code>IndexOutOfRangeException</code>	Tömbnél mindig <code>i < tomb.Length</code>
Pontosvessző a for után	<code>for (...);</code> → a ciklus üres, a törzs csak egyszer fut	Ne tegyél <code>;</code> -t a <code>for/while</code> záró zárójele után
Hiányzó <code>{ }</code>	Csak a ciklus utáni első sor ismétlődik, a többi nem	Több utasításnál mindig tegyél <code>{ }</code> -t
Elfelejtett nullázás	Összegzés előtt nem nullázod az <code>osszeg</code> változót	<code>int osszeg = 0;</code> a ciklus előtt

```
// Tömb: fix méretű
int[] jegyek = { 5, 4, 3, 5 };
Console.WriteLine(jegyek[0]);    // 5 (0-tól indexelünk!)
Console.WriteLine(jegyek.Length); // 4

// List: bármikor nő vagy csökken
List<string> nevek = new List<string>();
nevek.Add("Anna");
nevek.Add("Bence");
nevek.Add("Cili");
nevek.Remove("Anna");
Console.WriteLine(nevek.Count);  // 2

foreach (string n in nevek)
    Console.WriteLine(n);
```

- **Tömb**: fix darabszámú, azonos típusú elemek.
- **List<T>**: rugalmas lista, a **T** helyére az elem típusa kerül.
- Az index **0-tól** indul, az utolsó elem indexe **Count - 1**.
- Hibás index (**jegyek[10]**) → **IndexOutOfRangeException**.

A **List** használatához kellhet a **using System.Collections.Generic;** sor. Újabb projektekben ez már automatikus.

RÉSZLETES MAGYARÁZAT

- A **tömb** fix méretű, azonos típusú elemek sorozata. Méretét létrehozáskor adod meg (`new int[5]`), utána nem változik.
- Az új tömb elemei **alapértékkel** indulnak: számnál 0, `bool`-nál `false`, szövegnél és objektumnál `null`.
- A `List<T>` belül tömböt tart, és ha betelik, nagyobbra cseréli: ezért nő. Elem: `lista[0]`, méret: `Count` (tömbnél `Length`).
- A tömb és a lista **referenciatípus**: a `b = a` **nem másolat**, ugyanarra mutat. Másolat: `new List<int>(a)`.

ÍRATLAN SZABÁLYOK

- 1 Az index `0 ... Length-1`. Ellenőrizd, hogy a gyűjtemény **nem üres**, mielőtt `lista[0]`-t kérsz (`if (lista.Count > 0)`).
- 2 `First()` üres listán **kivételt dob**: használj `FirstOrDefault()`-ot vagy `Any()`-t.
- 3 A `RemoveAt(i)` hibás indexre kivételt dob, a `Remove(x)` hiányzó elemre csak `false`-t ad.
- 4 Ha a méret **nem változik**, használj tömböt; ha nő vagy csökken, `List`-et.

```
Random rnd = new Random();
int titok = rnd.Next(1, 101); // 1 és 100 között
int tipp = 0;
int probak = 0;

while (tipp != titok)
{
    Console.Write("Tipp (1-100): ");
    int.TryParse(Console.ReadLine(), out tipp);
    probak++;

    if (tipp < titok)
        Console.WriteLine("Több!");
    else if (tipp > titok)
        Console.WriteLine("Kevesebb!");
}
Console.WriteLine($"Eltaláltad {probak} tippből!");
```

- Ebben **minden eddigi dolog** benne van: változó, ciklus, feltétel, konverzió.
- **Random** = véletlenszám-generátor.
- Ha a játékos nem számot ír, a **TryParse** 0-t ad, így nem omlik össze.

Bővítés: számolj tippelési limitet, vagy kérdezd meg a végén, akar-e új játékot!

RÉSZLETES MAGYARÁZAT

- A program három fő része: **előkészítés** (véletlenszám, számlálók), **fő ciklus** (bekérés, ellenőrzés), **befejezés** (eredmény). Ez a szerkezet szinte minden konzolos játékban ugyanaz.
- A `rnd.Next(1, 101)` az alsó határt **tartalmazza**, a felsőt **nem**: 1–100 közötti szám.
- Az `int.TryParse(..., out tipp)` hibás bemenetnél 0-t ad, így a program nem omlik össze, csak „Több!” üzenetet kapsz.
- Az ellenőrzés három eset: kisebb, nagyobb, egyenlő. Az „egyenlő” ágat a `while` feltétele kezeli.

ÍRATLAN SZABÁLYOK

- 1 **Egyetlen** `Random` példány legyen: ciklusban újat létrehozva ugyanazt a számot kaphatod újra és újra.
- 2 Készülj a hibás bevitelre: üres sor, betű, negatív szám, túl nagy szám.
- 3 Ellenőrizd a **határokat**: `Next(1, 101)` → 1...100.
- 4 Adj **kiutat** (pl. max. próbálkozás), hogy a játék ne lehessen végtelen.

```
static int Osszead(int a, int b)
{
    return a + b;
}

static void Koszon(string nev)
{
    Console.WriteLine($"Szia, {nev}!");
}

static bool Paros(int szam)
{
    return szam % 2 == 0;
}

// A Main-ben:
int eredmeny = Osszead(3, 4);    // 7
Koszon("Anna");
Console.WriteLine(Paros(10));    // True
```

- Felépítés: **visszatérési típus + név + (paraméterek) + törzs.**
- A **return** ad vissza értéket, a **void** azt jelenti: **nem ad vissza semmit.**
- Egy metódus = egy feladat. Így a kód olvasható marad, és nem kell ismételtetni.
- A metódusokat a **Program** osztályba írd, a **Main** mellé.

RÉSZLETES MAGYARÁZAT

- A metódus **nevet ad egy kódrésznek**: a feladatot egyszer írod meg, és sokszor hívod. A hibát is egy helyen javítod.
- A **paraméter** a bemenet, a **visszatérési érték** a kimenet. A paraméter másolat: **int** esetén a külső változó nem változik, de lista/objektum esetén a **tartalom** módosulhat (referencia).
- A **void** metódus nem ad vissza semmit; nem-**void** metódusnál **minden útvonalon** kell **return**.
- A metódusnak **egy feladata** legyen, és a neve **igével** kezdődjön (**Mentes**, **Számol**).

ÍRATLAN SZABÁLYOK

- 1 Egy metódus = **egy feladat**. Ha 30 sornál hosszabb, vagy „és”-sel írnád le, bontsd szét.
- 2 Ellenőrizd a **paramétereket** az elején (null, tartomány): korai **return** vagy **throw**.
- 3 A **static** metódus **nem éri el** a nem-**static** mezőket: ne keverd a kettőt.
- 4 Rekurziónál (saját magát hívja) kell **kilépési feltétel**, különben **StackOverflowException**, amit nem lehet elkapni.
- 5 A metódus vagy **számol és visszaad**, vagy **kiír**: a kettőt ne keverd.

```
try
{
    Console.Write("Szám: ");
    int x = int.Parse(Console.ReadLine() ?? "");
    Console.WriteLine(100 / x);
}
catch (FormatException)
{
    Console.WriteLine("Ez nem szám!");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Nullával nem osztunk!");
}
catch (Exception ex)
{
    Console.WriteLine("Egyéb hiba: " + ex.Message);
}
finally
{
    Console.WriteLine("Ez mindig lefut.");
}
```

- A **try** blokkba az a kód kerül, ami **hibázhat**.
- Ha hiba (**kivétel**) történik, a program a megfelelő **catch** ágra ugrik, nem omlik össze.
- A **finally** mindig lefut, hiba esetén is.
- Sorrend: előbb a **konkrét**, a végén az általános **Exception**.

A fájlkezelésnél ez létfontosságú: fájl bármikor hiányozhat vagy zárolt lehet.

RÉSZLETES MAGYARÁZAT

- A **kivétel** (exception) egy objektum, amit a futtatókörnyezet „eldob”, ha valami váratlan történik. Ha senki nem kapja el, a **program összeomlik**.
- A **try**-ban a kód addig fut, amíg kivételt nem dob; ekkor a program azonnal a megfelelő **catch**-re ugrik, a **try** hátralévő része **kimarad**.
- A **catch** ágak sorrendje fontos: **konkrétabb előre**, az általános **Exception** a végére.
- A **finally mindig** lefut (siker, hiba, **return** esetén is): takarításra való, pl. fájl lezárása.

ÍRATLAN SZABÁLYOK

- 1 **Ne nyelj el hibát** üres **catch { }**-cel: legalább írd ki az **ex.Message**-t, különben sosem derül ki, mi romlott el.
- 2 Csak azt kapd el, amit **kezeln tudsz**. Amit nem, hagyd tovább menni, vagy **throw;**.
- 3 Ne használj kivételt **vezérlésre**: a **TryParse**, **File.Exists**, **if (b != 0)** tisztább és gyorsabb.
- 4 A **try** blokk legyen **kicsi**: csak a kockázatos sorok kerüljenek bele.
- 5 **throw;** megtartja a hiba helyét, **throw ex;** törli: az előbbit használd.

A program memóriája **törlődik**, amikor bezárod. Ha adatot meg akarsz tartani (mentés, napló, beállítás), **fájlba** kell írnod. Kell hozzá: `using System.IO;`

Metódus	Mit csinál?
<code>File.WriteAllText(út, szöveg)</code>	Fájlba ír (új fájlt hoz létre, vagy felülírja a régit)
<code>File.ReadAllText(út)</code>	Az egész fájlt egy szöveggént beolvassa
<code>File.AppendAllText(út, szöveg)</code>	A fájl végére fűz , a régi tartalom megmarad
<code>File.WriteAllLines(út, sorok)</code>	Egy tömb/lista elemeit sorokként írja ki
<code>File.ReadAllLines(út)</code>	Soronként beolvassa, <code>string[]</code> -t ad
<code>File.Exists(út) / File.Delete(út)</code>	Létezik-e a fájl? / Törlés

RÉSZLETES MAGYARÁZAT

- A **memória (RAM)** a program bezárásakor törlődik, a **fájl** a háttértáron marad. Fájlba írni = adatot **tartósan** menteni.
- A fájlhoz két dolog kell: **útvonal** (hol van) és **tartalom**. A `System.IO` névtér adja az eszközöket.
- A `File` osztály statikus metódusai kényelmesek (egy sor), de az **egész fájlt a memóriába** töltik: nagy fájlnál használj `StreamReader`-t.
- A fájlműveletek a **külvilággal** dolgoznak, ami bármikor megváltozhat: nincs fájl, nincs jogosultság, tele a lemez, másik program használja.

ÍRATLAN SZABÁLYOK

- 1 Olvasás előtt **mindig** `File.Exists` + `try/catch`.
- 2 Írás előtt dönts el: **felülír vagy hozzáfűz?** (`WriteAll...` ↔ `Append...`).
- 3 **Jelszót** és érzékeny adatot ne tárolj sima szöveges fájlban.
- 4 Tesztelj **másolaton**, ne az egyetlen fontos fájlodon.

```
using System.IO;

// Írás (új fájl, vagy felülírás)
File.WriteAllText("adat.txt", "Első sor");

// Olvasás
string tartalom = File.ReadAllText("adat.txt");
Console.WriteLine(tartalom);           // Első sor

// Hozzáadás a végéhez
File.AppendAllText("adat.txt", "\nÚj sor");

Console.WriteLine(File.ReadAllText("adat.txt"));
// Első sor
// Új sor
```

- Ez a **legegyszerűbb** módszer, kis fájlokhoz tökéletes.
- A `\n` új sort jelent a szövegben.
- A **"adat.txt" relatív útvonal**: a program mappájában keresi/hozza létre (részletek: az „Útvonalak” dia).

Vigyázz: a `WriteAllText` minden hívásnál **felülírja** a fájlt! Hozzáadáshez az `AppendAllText` kell.

RÉSZLETES MAGYARÁZAT

- A `WriteAllText` **megnyitja** a fájlt, kiírja a szöveget és **lezárja**: egyetlen hívás. Ha a fájl létezik, a **régi tartalom elvész**.
- A `ReadAllText` az **egész** fájlt egyetlen `string`-be olvassa, a sortörésekkel (`\n` vagy `\r\n`) együtt.
- Az alapértelmezett kódolás **UTF-8**: az ékezetes betűk (ő, ú) rendben vannak, ha írásnál és olvasásnál ugyanazt használod.
- Az `AppendAllText` a fájl **végére** ír; ha a fájl nincs, létrehozza.

ÍRATLAN SZABÁLYOK

- 1 A `WriteAllText` előtt gondold végig, kell-e a régi tartalom: **nincs visszavonás**.
- 2 Új sorhoz használd az `Environment.NewLine`-t, így Windows és Linux is helyes.
- 3 Gigabájtos fájlt ne olvass be `ReadAllText`-tel: `OutOfMemoryException`.
- 4 Íráskor győződj meg róla, hogy a **mappa létezik** (`Directory.Exists`).

```
string[] nevek = { "Anna", "Bence", "Cili" };  
File.WriteAllLines("nevek.txt", nevek);  
  
string[] beolvasott = File.ReadAllLines("nevek.txt");  
foreach (string sor in beolvasott)  
    Console.WriteLine(sor);  
  
if (File.Exists("nevek.txt"))  
    Console.WriteLine("A fájl létezik.");  
else  
    Console.WriteLine("Nincs ilyen fájl.");  
  
File.Delete("nevek.txt");    // törlés
```

- Listát is kiírhatsz:
`WriteAllLines("f.txt", lista).`
- A `ReadAllLines` után `foreach`-cel egyenként feldolgozhatod a sorokat.
- Olvasás előtt **mindig** ellenőrizd a `File.Exists`-szel, hogy van-e fájl!

RÉSZLETES MAGYARÁZAT

- A `ReadAllLines` a fájlt **sorokra bontja**, `string[]`-t ad. A sor végi sortörés **nem** kerül bele.
- A `WriteAllLines` minden elem végére sortörést tesz, és **felülírja** a fájlt.
- A sorokat gyakran fel kell dolgozni: `sor.Split(';')`, `int.TryParse`. A sor felépítésében **előre meg kell állapodni** (elválasztó, mezők száma).
- A fájl végén gyakran van **üres sor**: a `Split` ilyenkor 1 elemet ad, és az `r[2]` **`IndexOutOfRangeException`** hibát dob.

ÍRATLAN SZABÁLYOK

- 1 Feldolgozás előtt: `if (string.IsNullOrEmpty(sor)) continue;`
- 2 A `Split` után nézd meg a **mezők számát** (`r.Length == 3`).
- 3 Az elválasztó (`;`) **ne szerepelhessen** az adatban: különben elcsúszik a feldolgozás.
- 4 A `File.Delete` **véglegesen töröl** (nem megy a Lomtárba): törlés előtt kérj megerősítést.

```
using (StreamWriter iro = new StreamWriter("naplo.txt", append: true))
{
    iro.WriteLine("A program elindult: " + DateTime.Now);
    iro.WriteLine("Második bejegyzés");
}

using (StreamReader olvaso = new StreamReader("naplo.txt"))
{
    while (!olvaso.EndOfStream)
    {
        string sor = olvaso.ReadLine() ?? "";
        Console.WriteLine(sor);
    }
}
```

- Nagyobb fájlnál vagy ha **sorról sorra** akarsz dolgozni, ez a módszer.
- A **using** blokk a végén **automatikusan lezárja** a fájlt. Ez nagyon fontos!
- **append: true** = hozzáfűz, **false** (alap) = felülír.

Ha lezáratlanul marad egy fájl, később „**a fájlt másik folyamat használja**” hibát kapsz.

RÉSZLETES MAGYARÁZAT

- A `StreamWriter/StreamReader` a fájlt **folyamként** kezeli: megnyitja, soronként dolgozik, majd lezárja. Nem tölt be mindent a memóriába.
- A `using` blokk garantálja, hogy a végén lefut a `Dispose()`, vagyis a fájl **lezáródik**, akkor is, ha közben kivétel történik.
- A `StreamWriter` **pufferel**: az adat csak lezáráskor (vagy `Flush`-kor) kerül biztosan a lemezre. Ezért veszhet el adat, ha a program a lezárás előtt összeomlik.
- Rövidebb forma (újabb C#): `using var iro = new StreamWriter("naplo.txt");`; a blokk végén zár.

ÍRATLAN SZABÁLYOK

- 1 **Mindig `using`**: lezáratlan fájl → „a fájlt másik folyamat használja” hiba, vagy hiányzó adat.
- 2 Ugyanazt a fájlt **ne nyisd meg kétszer** írásra egyszerre.
- 3 Olvasás közben ugyanazt a fájlt **ne írd**: előbb olvass be, aztán írd.
- 4 A `ReadLine()` eredményét kezeld null-ként (`?? ""`).

```
// Hol keres a program?  
Console.WriteLine(Directory.GetCurrentDirectory());  
// pl. ...\\projekt\\bin\\Debug\\net8.0  
  
// Teljes (abszolút) útvonal: @ jel kell a \\ miatt  
string fajl = @"C:\\Users\\Anna\\Documents\\adat.txt";  
  
// Biztonságos összefűzés  
string mappa = @"C:\\Adatok";  
string ut = Path.Combine(mappa, "adat.txt");  
  
// Mappa létrehozása, ha nincs  
if (!Directory.Exists(mappa))  
    Directory.CreateDirectory(mappa);
```

- **Relatív** útvonal ("adat.txt"): a futó program mappájából indul, ez általában a bin\\Debug\\... mappa.
- **Abszolút** útvonal: a teljes cím a meghajtótól.
- A \\ speciális karakter a szövegben, ezért írd elé @-ot, vagy duplázd meg: "C:\\\\Adatok".
- A **Path.Combine** helyesen teszi össze az útvonalat.

RÉSZLETES MAGYARÁZAT

- **Relatív útvonal:** a „munkakönyvtárhoz” viszonyít (Visual Studióban általában `bin\Debug\net8.0`). **Abszolút:** a meghajtótól megadott teljes cím.
- A Windows `\`-t, a Linux/Mac `/`-t használ. A `Path.Combine` mindig a megfelelőt teszi.
- A `\` a C# szövegben **escape-karakter** (`\n`, `\t`): a `"C:\Users"` hibás. Írd `@"C:\Users"`, vagy `"C:\\Users"`.
- Felhasználói mappákhoz: `Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments)`: nem kell a felhasználónevet beleírni.

ÍRATLAN SZABÁLYOK

- 1 **Ne égess be** a géphez kötött útvonalat (`C:\Users\Anna\...`): máshol nem fog működni.
- 2 Útvonalat ne fűzz `+`-szal: használd a `Path.Combine(mappa, fajl)`-t.
- 3 Ha nem találod a fájlt, írd ki a `Directory.GetCurrentDirectory()`-t.
- 4 A `C:\` gyökérbe és a `Program Files`-ba **nem írhatsz** (`UnauthorizedAccessException`): használd a `Documents` vagy az `AppData` mappát.

Hibaüzenet / tünet	Miért van?	Javítás
<code>FileNotFoundException</code>	A fájl nincs ott, ahol keresed (rossz név vagy mappa)	<code>File.Exists</code> ellenőrzés; írd ki a <code>GetCurrentDirectory()</code> -t
<code>DirectoryNotFoundException</code>	A mappa nem létezik	<code>Directory.CreateDirectory(...)</code> előbb
<code>UnauthorizedAccessException</code>	Nincs jogod oda írni (pl. <code>C:\</code> gyökér)	Írj a Documents vagy a program mappájába
<code>IOException</code> (használatban)	Másik program vagy lezáratlan stream fogja	<code>using</code> blokk; zárd be a fájlt Jegyzettömbben
Elveszett az adat	<code>WriteAllText</code> felülírt	<code>AppendAllText</code> vagy <code>append: true</code>
Ékezetek elromlanak (ő, ú)	Rossz kódolás	<code>File.WriteAllText(út, szöveg, Encoding.UTF8); using System.Text;</code>

```
string fajl = "jegyzetek.txt";

while (true)
{
    Console.WriteLine("1 - Új jegyzet | 2 - Lista | 0 - Kilépés");
    string valasz = Console.ReadLine() ?? "";

    if (valasz == "1")
    {
        Console.Write("Jegyzet: ");
        string uj = Console.ReadLine() ?? "";
        File.AppendAllText(fajl, uj + Environment.NewLine);
    }
    else if (valasz == "2")
    {
        if (File.Exists(fajl))
            Console.WriteLine(File.ReadAllText(fajl));
        else
            Console.WriteLine("Még nincs jegyzet.");
    }
    else if (valasz == "0")
    {
        break;
    }
}
```

- Összekapcsolja a **ciklust**, **feltételeket** és a **fájlkezelést**.
- A **while (true)** végtelen ciklus, a **break** lép ki belőle.
- A jegyzetek a **program bezárása után is megmaradnak**.

Bővítés: jegyzet törlése, számozott lista (**ReadAllLines** + **for**), keresés a jegyzetek közt.

RÉSZLETES MAGYARÁZAT

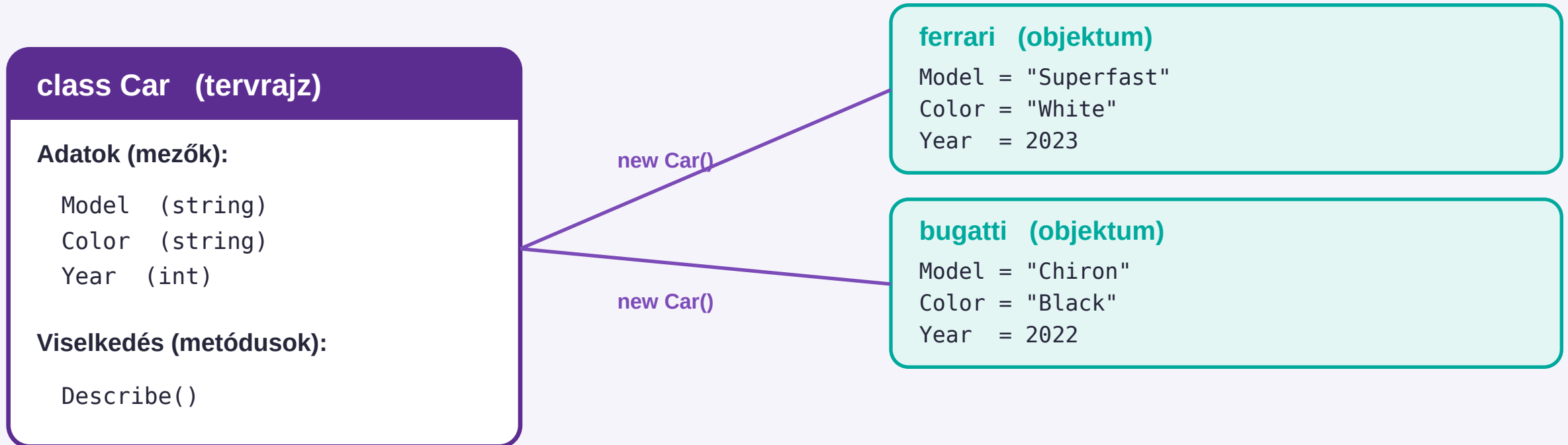
- A program egy egyszerű **állapotgép**: a menüben „áll”, és a felhasználó parancsa viszi át másik állapotba (hozzáad, listáz, kilép).
- Az `AppendAllText` miatt minden jegyzet a fájl végére kerül, így az **adat a futások között is megmarad**.
- A `while (true) + break` minta: a kilépési feltétel a ciklus belsejében van (a `0` ág).
- Az `Environment.NewLine` sortörést fűz, ezért az új jegyzetek **egymás alá** kerülnek.

ÍRATLAN SZABÁLYOK

- 1 Minden menüpontnál **kezel** a **váratlan választ** (`else` ág: „Ismeretlen parancs”).
- 2 Üres jegyzetet ne fogadj el (`string.IsNullOrEmpty`).
- 3 Ments **minden változás után**, ne csak kilépéskor: ha a program összeomlik, nem vész el az adat.
- 4 A programnak a **hiányzó fájlal is működnie kell** (első indítás): ezt a `File.Exists` intézi.

Osztályok: mire jók, és hogyan működnek?

7 · OSZTÁLYOK



- Az **osztály** egy **tervrajz**: leírja, milyen adatai és tudásai lesznek valaminek.
- Az **objektum** a tervrajzból gyártott **konkrét példány**. Belőle annyit csinálhatsz, amennyit akarsz.

Osztályok nélkül minden autóhoz külön változó kellene (**ferrariModel**, **bugattiModel**...). Osztállyal minden **egy csomagban** van.

RÉSZLETES MAGYARÁZAT

- Az **osztály** új **típus**, amit te definiálsz. Ahogy az `int`, úgy a `Car` is típus: változót hozhatsz létre belőle.
- Az **objektum** az osztály egy **példánya**, a memóriában él. A változó (`ferrari`) csak **hivatkozást** (referenciát) tart rá.
- Az osztály tartalma: **mezők/tulajdonságok** (adatok), **metódusok** (viselkedés), **konstruktor** (létrehozás).
- Két objektum **független**: a `ferrari.Year` módosítása nem érinti a `bugatti.Year`-t. De a `Car x = ferrari;` **nem másolat**: mindkettő ugyanarra az objektumra mutat.

ÍRATLAN SZABÁLYOK

- 1 Az osztály neve **főnév**, nagy kezdőbetűvel (`Car`, `Konyv`), és **egy dolgot** modellezzen.
- 2 `Car a = null;` után az `a.Model` **NullReferenceException**: használat előtt `new`, vagy `if (a != null)`.
- 3 A `b = a` hozzárendeléssel **nem másolsz**: ha az egyiket módosítod, a másik is változik.
- 4 Ha a program nő, tedd az osztályt **külön fájlba** (`Car.cs`).

```
internal class Program
{
    static void Main(string[] args)
    {
        Car ferrari = new Car();
        ferrari.Model = "Superfast";
        ferrari.Color = "White";
        ferrari.Year = 2023;

        Car bugatti = new Car();
        bugatti.Model = "Chiron";
        bugatti.Color = "Black";
        bugatti.Year = 2022;

        Console.WriteLine(ferrari.Model);
        Console.WriteLine(bugatti.Model);
    }
}

public class Car
{
    public string Model;
    public string Color;
    public int Year;
}
```

- A `Car` és a `Program` két külön osztály. Mindkettő saját `{ }` blokkban van.
- A `new Car()` új objektumot hoz létre.
- A `public` kell, hogy a `Program` is lássa a mezőket.
- Az objektum adataihoz **ponttal** férsz hozzá: `ferrari.Model`.

Gyakori hiba: ha a `Main` véletlenül a `Car` kapcsos zárójelén belül marad, akkor is lefut, de rossz szerkezet. Mindig számold meg, hogy minden `{`-nek van-e `}` párja!

RÉSZLETES MAGYARÁZAT

- A `new Car()` **memóriát foglal** egy új autónak, és visszaad egy hivatkozást; ezt tárolja a `ferrari` változó.
- A mezők kezdőértéke **alapérték** (`null`, 0), amíg be nem állítod: így `ferrari.Model.Length` az elején **`NullReferenceException`**.
- A `public` mező bárhonnán módosítható: kényelmes, de nincs ellenőrzés (lehetne `Year = -5`). Ezért használunk később **tulajdonságot**.
- A `Main` és az osztály egymás mellett él; csak a `{ }` hibás zárása miatt kerülhet véletlenül az egyik a másikba.

ÍRATLAN SZABÁLYOK

- 1 Ne kerüljön a `Main` az osztályba véletlenül: nézd meg, hogy az osztály `}`-ja a `Main` előtt van-e.
- 2 Mezőt ne tegyél `public`-ra, ha nem muszáj: használj tulajdonságot.
- 3 A szöveges mezőket inicializáld (`= ""`), hogy ne legyenek `null`-ok.
- 4 Ne legyen két azonos nevű osztály ugyanabban a névtérben.

```
public class Car
{
    public string Model { get; set; }
    public string Color { get; set; }
    public int Year { get; set; }

    // Konstruktor: a "new Car(...)" ezt futtatja
    public Car(string model, string color, int year)
    {
        Model = model;
        Color = color;
        Year = year;
    }
}

// Használat:
Car ferrari = new Car("Superfast", "White", 2023);
Car bugatti = new Car("Chiron", "Black", 2022);
Console.WriteLine(ferrari.Model);
```

- **Konstruktor:** egy különleges metódus, **az osztály nevét viseli**, és létrehozáskor fut le. Így nem maradhat üresen egy adat.
- **Tulajdonság** (`{ get; set; }`): a mező „szebb” változata. Ez a szokásos C# módszer. Nagy kezdőbetűvel írjuk.

Ha írsz konstruktort, a `new Car()` (paraméter nélkül) már **nem működik**, mindig meg kell adni az adatokat.

RÉSZLETES MAGYARÁZAT

- A **konstruktor** a `new` kulcsszóval hívódik, **nincs visszatérési típusa**, neve az osztályével megegyezik. Feladata: az objektumot **érvényes állapotba** hozni.
- Ha nem írsz konstruktort, a C# ad egy paraméter nélkülit. Ha **írsz sajátot**, az automatikus **megszűnik**.
- A **tulajdonság** (`{ get; set; }`) mögött a fordító rejtett mezőt hoz létre; később a `get/set` tartalmát cserélheted, a használó kód nem változik.
- Lehet **több** konstruktor is (túlterhelés), különböző paraméterekkel.

ÍRATLAN SZABÁLYOK

- 1 A konstruktor után az objektum **használható** legyen: minden kötelező adat bekerül.
- 2 Ellenőrizd a paramétereket (null, tartomány), és dobj `ArgumentException`-t.
- 3 Ha a szülőosztálynak nincs paraméter nélküli konstruktora, a leszármazottnak **kell** a `base(...)`.
- 4 A konstruktorban **ne végezz hosszú műveletet** (fájl, hálózat): lassú, és hibára futhat.

```
public class Car
{
    public string Model { get; set; }
    public int Year { get; set; }
    public int Speed { get; private set; }

    public Car(string model, int year)
    {
        Model = model;
        Year = year;
    }

    public void Gyorsit(int mennyit)
    {
        Speed += mennyit;
    }

    public override string ToString()
    {
        return $"{Year} {Model} ({Speed} km/h)";
    }
}
```

- Az osztály nemcsak **adatot**, hanem **viselkedést** is tárol: `a.Gyorsit(100)`.
- A metódus az objektum **saját adataival** dolgozik.
- A `ToString` felülírásával a `Console.WriteLine(a)` szépen kiírja az objektumot.
- `private set`: kívülről csak olvasható, belül módosítható.

Használat: `Car a = new Car("Chiron", 2022);`
majd `a.Gyorsit(100);` és
`Console.WriteLine(a);` → 2022 Chiron (100 km/h)

RÉSZLETES MAGYARÁZAT

- Az osztály metódusa **az objektum saját adataival** dolgozik: a **Gyorsít** a **Speed** tulajdonságot módosítja. A **this** az aktuális objektum (általában elhagyható).
- A **private set** miatt kívülről nem módosítható az érték, csak metóduson át: így a **szabály egy helyen** van.
- A **ToString()** minden objektum öröklött metódusa; felülírva (**override**) te határozod meg, hogyan jelenjen meg szöveggént. Ezt hívja a **Console.WriteLine(a)** és a **(\$"{a}")**.
- A metódus vagy **állapotot változtat** (**Gyorsít**), vagy **kérdez** (**Kor**): a kettő különüljön el.

ÍRATLAN SZABÁLYOK

- 1 A metódus ne hagyja az objektumot **érvénytelen állapotban** (pl. **Speed < 0**): ellenőrizz belül.
- 2 A **ToString** ne dobjon kivételt, és ne legyen lassú: a hibakereső is hívja.
- 3 Ha a metódus nem használ objektumadatot, lehet **static**.
- 4 Nevek: ige a műveletnek (**Gyorsít**), **Is.../Van...** a logikai kérdésnek.

```
public class Car
{
    private int _year;

    public int Year
    {
        get { return _year; }
        set
        {
            if (value < 1886)
                throw new ArgumentException("Túl korai év!");
            _year = value;
        }
    }
}

// Használat:
Car a = new Car();
a.Year = 2023;    // rendben
a.Year = 1500;    // kivételt dob!
```

- **public**: bárhonnan elérhető. **private**: csak az osztályon belül látszik.
- Az **egységbezárás** (encapsulation) lényege: az osztály maga **őrzi az adatai épségét**.
- A **set** ágban ellenőrizhetjük az új értéket, és ha rossz, hibát dobunk.
- A **value** a beállítani kívánt új érték.

Ökölszabály: a mező legyen **private**, amit kívülről is kell látni, azt tedd **public** tulajdonsággá.

RÉSZLETES MAGYARÁZAT

- **Egységbezárás** (encapsulation): az adat a saját osztályában védett, kívülről csak engedélyezett módon módosítható. Így az objektum mindig **érvényes** marad.
- A **set** ágban a **value** a beállítani kívánt új érték. Itt ellenőrizhetsz, és **kivételt dobhatsz**, ha érvénytelen.
- Láthatóságok: **public** (mindenhol), **private** (csak az osztályon belül), **protected** (osztály és leszármazottai), **internal** (azonos projekt).
- Alapértelmezés: az osztály tagjai **private**, az osztályok pedig **internal**.

ÍRATLAN SZABÁLYOK

- 1 Mindent **private**-nek kezdj, és csak akkor nyisd ki, ha kell: **minimális láthatóság**.
- 2 A kivételüzenet legyen **érthető**: "Az **évszám** **1886 és 2100 között lehet.**"
- 3 A **set**-ben a **mezőt** (**_year**) írd, ne a tulajdonságot (**Year = value**): végtelen rekurzió és **StackOverflow**.
- 4 A kivételt a **hívó** kapja el (**try/catch**), ne az osztály nyelje el.

```
public class Car
{
    // Model, Color, Year és a konstruktor: mint eddig

    public virtual void Describe()
    {
        Console.WriteLine($"{Year} {Color} {Model}");
    }
}

public class ElectricCar : Car
{
    public int BatteryKwh { get; set; }

    public ElectricCar(string model, string color, int year, int kwh)
        : base(model, color, year)
    {
        BatteryKwh = kwh;
    }

    public override void Describe()
    {
        base.Describe();
        Console.WriteLine($"    Akku: {BatteryKwh} kWh");
    }
}
```

- `: Car` azt jelenti: az `ElectricCar` egy `Car`, plusz saját extrái.
- A `base(...)` a szülő konstruktorát hívja meg.
- `virtual` a szülőben = „felülírható”, `override` a gyerekekben = „én máshogy csinálom”.
- Így minden típus a saját módján csinálja ugyanazt (**polimorfizmus**).

RÉSZLETES MAGYARÁZAT

- Az öröklődés „X **egy** Y” viszony (**ElectricCar** egy **Car**). A gyermekosztály a szülő minden **public/protected** tagját megkapja, és bővítheti.
- A **virtual** metódust a leszármazott **override**-dal **felülírhatja**. **override** nélküli azonos nevű metódus csak **elrejt**i a szülőét (figyelmeztetés).
- **Polimorfizmus**: **List<Car>**-ban lehet **ElectricCar**, és az **a.Describe()** a **tényleges** típus metódusát hívja.
- A **base.Describe()** a szülő változatát hívja, így nem kell megismételni a kódot.

ÍRATLAN SZABÁLYOK

- 1 Csak akkor örökíts, ha tényleg „X **egy** Y”. Ha „X-nek **van** Y-ja”, használj tartalmazást (**Car Engine** mezővel).
- 2 A hierarchia ne legyen 2–3 szintnél mélyebb.
- 3 A leszármazott konstruktora hívja a szülőét: **: base(...)**.
- 4 Típusellenőrzésre használd az **is/as**-t (**if (a is ElectricCar e)**): a rossz (**ElectricCar**)a **InvalidCastException**-t dob.

```
using System.Linq;

List<Car> garazs = new List<Car>
{
    new Car("Superfast", "White", 2023),
    new Car("Chiron", "Black", 2022),
    new ElectricCar("Taycan", "Blue", 2024, 93)
};

foreach (Car a in garazs)
    a.Describe();           // mindegyik a saját módján

var ujak = garazs.Where(a => a.Year >= 2023);
var rendezett = garazs.OrderBy(a => a.Year);
int db = garazs.Count(a => a.Color == "Black");

foreach (Car a in ujak)
    Console.WriteLine(a.Model);
```

- A lista **bármilyen típusú** elemet tárolhat, akár saját osztályt is.
- Egy `List<Car>`-ba az `ElectricCar` is bekerülhet, mert az is `Car`.
- **LINQ**: szűrés, rendezés, számolás egy sorban. A `a => ...` a „lambda”: *, „minden `a`-ra, ami...”*.

Feladat: Írj `Person` osztályt, aminek van egy `List<Car> Autok` tulajdonsága, és `BuyCar(Car)` meg `ListCars()` metódusa!

RÉSZLETES MAGYARÁZAT

- A `List<Car>` az objektumokra mutató **hivatkozásokat** tárol. Ha módosítod a `garazs[0].Color`-t, az az objektumot változtatja, bárhonnan hivatkozol rá.
- A **LINQ** (`Where`, `OrderBy`, `Count`, `Select`, `Any`, `First`) adatot kérdez le **deklaratíván**: leírod, mit akarsz, nem azt, hogyan.
- A LINQ **halasztott végrehajtású**: a `Where(...)` csak recept, igazán akkor fut, amikor bejárod (`foreach`, `ToList()`, `Count()`).
- A lambda `a => a.Year >= 2023` egy mini névtelen metódus: az `a` az aktuális elem.

ÍRATLAN SZABÁLYOK

- 1 `First()` és `Single()` **üres találatnál kivételt dobnak**: használj `FirstOrDefault()`-ot (és ellenőrizd null-ra) vagy `Any()`-t.
- 2 Ha az eredményt többször használod, tedd `ToList()`-be: különben többször lefut a lekérdezés.
- 3 A LINQ bejárása közben **ne módosítsd** az eredeti listát.
- 4 Régebbi projektben kell a `using System.Linq`; sor.

```
static void Mentos(List<Car> autok, string fajl)
{
    List<string> sorok = new List<string>();
    foreach (Car a in autok)
        sorok.Add($"{a.Model};{a.Color};{a.Year}");

    File.WriteAllLines(fajl, sorok);
}

static List<Car> Betoltes(string fajl)
{
    List<Car> autok = new List<Car>();
    if (!File.Exists(fajl))
        return autok;           // nincs fájl: üres lista

    foreach (string sor in File.ReadAllLines(fajl))
    {
        string[] r = sor.Split(';');
        autok.Add(new Car(r[0], r[1], int.Parse(r[2])));
    }
    return autok;
}
```

- Itt jön össze **minden**: osztály, lista, szöveg, ciklus, metódus és fájlkezelés.
- **Mentés**: minden autót egy sorra alakítunk, az adatokat **;** választja el (ez a **CSV** formátum).
- **Betöltés**: soronként beolvasunk, `Split(';')`-vel szétvágunk, és az adatokból új **Car** objektumot építünk.
- Mindkét metódust a **Program** osztályba írd, a **Main** mellé.

RÉSZLETES MAGYARÁZAT

- Az objektumot nem írhatjuk közvetlenül fájlba: **szöveggé alakítjuk** (`Model;Color;Year`), betöltéskor pedig fordítva. Ez a szerializálás és deszerializálás.
- A `Betoltes` először ellenőrzi, létezik-e a fájl; ha nem, **üres listát** ad, így az első indításkor sem omlik össze.
- A `Split(';')` mezőkre bontja a sort, az `int.Parse(r[2])` az évet számmá alakítja.
- Ez a formátum (CSV) egyszerű; összetettebb adatnál a **JSON** jobb (`System.Text.Json`).

ÍRATLAN SZABÁLYOK

- 1 Betöltéskor **minden sort ellenőrizz**: üres, hiányzó mező (`r.Length`), hibás szám (`TryParse`). Egy rossz sor ne vigye el az egész programot.
- 2 Ments **minden változás után**, ne csak a végén.
- 3 A fájlnevet ne ismételd: tedd **konstansba** (`const string Fajl = "autok.txt";`).
- 4 Felülírás előtt készíts **biztonsági másolatot** (`File.Copy`).
- 5 Ne tárolj `;`-t tartalmazó adatot ezzel a formátummal.

```
static void Main(string[] args)
{
    string fajl = "autok.txt";

    List<Car> garazs = Betoltes(fajl);
    Console.WriteLine($"Betöltve: {garazs.Count} autó");

    garazs.Add(new Car("Superfast", "White", 2023));
    Mentos(garazs, fajl);

    foreach (Car a in garazs)
        Console.WriteLine($"{a.Year} {a.Color} {a.Model}");
}
```

- Minden futtatásnál **betölti** az előzőleg mentett autókat, hozzáad egyet, és **elmenti** az egészet.
- Futtasd többször: a garázs **minden alkalommal nő**. Az adat a program bezárása után is megmarad.
- Nyisd meg az **autok.txt**-t a **bin\Debug\...** mappában, és nézd meg, hogyan néz ki belülről!

Hibatűrőbb verzió: tedd a **Betoltes** hívást **try/catch** blokkba, és a soroknál ellenőrizd az **r.Length == 3** feltételt.

RÉSZLETES MAGYARÁZAT

- A **Main** a **vezénylő**: maga alig csinál valamit, a metódusokat hívja a megfelelő sorrendben: betölt, módosít, ment, kiír.
- Az **adat útja**: fájl → **Betöltés** → lista (memória) → módosítás → **Mentes** → fájl. A lista csak a futás alatt él, a fájl tartós.
- A példában minden futtatásnál bővül a lista: ez **szándékos**, hogy lásd, hogy a mentés működik.
- Ha az **autok.txt** sérült, a **Betöltés** kivételt dobhat: ezt a **Main**-ben **try/catch**-cel kell kezelni.

ÍRATLAN SZABÁLYOK

- 1 A **Main** legyen **rövid**: a részletek menjenek metódusokba.
- 2 A fájlnev **egy** helyen szerepeljen.
- 3 A program **ne omoljon össze** hibás fájlról: kapd el, írd ki a hibát, és kezdj üres listával.
- 4 **Felülírás előtt győződj meg róla, hogy a betöltés sikerült**: különben egy üres listával felülírod a jó fájlt!

A 16 arany szabály: hogy ne legyen véletlen összeomlás

ÖSSZEFOGLALÓ

1 Felhasználói bevitelre **mindig** `TryParse`, ne `Parse`.

2 Ne feltételezd, hogy egy érték **nem null**, vagy hogy a lista **nem üres**.

3 Tömbnél a ciklus feltétele `i < Length`, soha `<=`.

4 Minden ciklusnak legyen **megállási feltétele**.

5 **Osztás előtt** ellenőrizd, hogy az osztó nem nulla.

6 Fájlt `using`-gal nyiss, olvasás előtt `File.Exists` és `try/catch`.

7 **Ne módosíts gyűjteményt** `foreach` közben.

8 Ne írd **üres** `catch`-et: legalább naplózd a hibát.

9 Íráskor dönts el: **felülír vagy hozzáfűz**?

10 Relatív útvonal a `bin\Debug` mappába mutat; ne égecs be abszolút útvonalat.

11 Pénzhez `decimal`, soha `double`.

12 Egy metódus = **egy feladat**, és az elején ellenőrizd a paramétereket.

13 Ami nem kell kívülről, legyen `private`.

14 `==` összehasonlít, `=` értéket ad; **ne tegyél** `;-t if/for/while` után.

15 **Ments és építs gyakran**, olvasd a Hibalistát felülről.

16 **Felülírás előtt győződj meg a betöltés sikeréről.**

- **Interfészek** (`interface`) és **absztrakt osztályok** (`abstract`): közös „szerződések” osztályoknak.
- **Dictionary<K,V>**, **enum**, **static**, `struct`: további hasznos eszközök.
- **JSON mentés** (`System.Text.Json`): adatok korszerűbb fájlformátumban.
- **Grafikus felület**: WinForms vagy WPF, ha ablakos programot szeretnél.
- **Weboldal** ASP.NET-tel, **játék** Unity-vel: itt mindenhol az osztályok a főszereplők.

Ingyenes forrás: **Microsoft Learn** C# kezdő útvonala (learn.microsoft.com/dotnet/csharp). A legjobb tanulás: építs saját kis projektet!

Gyakorlás: ciklusok (alap szint)

GYAKORLÁS

#	Feladat	Tipp
1	Írd ki a számokat 1-től 20-ig for ciklussal.	<code>for (int i = 1; i <= 20; i++)</code>
2	Írd ki a számokat 10-től 1-ig visszafelé while ciklussal.	Indulj <code>n = 10</code> -ről, a törzsben <code>n--</code>
3	Kérj be egy N számot, és írd ki az 1 + 2 + ... + N összeget.	Kell egy <code>osszeg = 0</code> a ciklus előtt
4	Kérj be jelszót do-while -al addig, amíg a felhasználó be nem írja: titok .	Feltétel: <code>while (jelszo != "titok")</code>
5	Írd ki a páros számokat 2-től 50-ig , először <code>i += 2</code> -vel, aztán <code>%</code> és <code>continue</code> használatával.	Páros, ha <code>i % 2 == 0</code>
6	Kérj be egy szöveget, és írd ki ötször egymás alá for ciklussal.	Kiírás a törzsben

Hogyan kezdj hozzá? 1) Írd le magyarul, mit kell csinálnia. 2) Döntsd el, melyik ciklus kell. 3) Írd meg, futtasd, próbáld ki hibás bemenettel is.

Gyakorlás: ciklusok (közép és haladó)

GYAKORLÁS

#	Feladat	Tipp
7	Kérj be számokat 0-ig . Írd ki az összegüket , darabszámukat és átlagukat .	<code>while (szam != 0);</code> osztás előtt nézd meg: <code>db > 0</code>
8	FizzBuzz : 1–100-ig írd ki a számokat, de 3-mal oszthatónál Fizz , 5-tel oszthatónál Buzz , mindkettővel oszthatónál FizzBuzz legyen.	Előbb a 15-tel oszthatót vizsgáld
9	Számold meg, hány 3-mal vagy 5-tel osztható szám van 1 és 1000 között.	Számláló változó + <code> </code>
10	Írd ki a szorzótáblát 1-től 10-ig egymásba ágyazott ciklussal.	Külső: sorok, belső: oszlopok
11	Rajzolj csillagokból piramist (1, 3, 5, 7, 9 csillag, középre igazítva).	<code>new string('*', 3)</code> ismételt karaktert
12	Írd ki a Fibonacci-sorozat első 20 elemét (1, 1, 2, 3, 5, 8...).	Két változó: <code>a</code> , <code>b</code> ; új = <code>a + b</code>
13	Írj metódust, ami megmondja egy számról, prím-e . Írd ki a prímeket 1–100 között.	Oszd el 2-től $\sqrt{n}$ -ig, <code>%</code> -ot használj

Gyakorlás: változók, döntések, listák, metódusok

GYAKORLÁS

Téma	Feladat
Változók	Kérd be a nevedet és a születési évedet , és írd ki, hány éves vagy idén (<code>DateTime.Now.Year</code>).
if / else	Kérj be egy számot: negatív, nulla vagy pozitív ? Páros vagy páratlan? Mindkettőt írd ki.
switch	Kérj be egy számot 1–12 között, és írd ki a hónap nevét . Érvénytelen szám esetén írd hibaüzenetet.
Listák	Tárolj egy <code>List<int></code> -ben számokat (0-ig kérj be), majd írd ki a legnagyobbat, a legkisebbet és az átlagot .
Metódusok	Írd <code>Faktorialis(int n)</code> metódust ($5! = 120$), és egy <code>Max(int a, int b)</code> metódust.
Hibakezelés	Kérj be egy számot <code>int.Parse</code> -szal, és kezeld <code>try/catch</code> -cel a hibás bemenetet. Addig kérj újra, amíg nem jó.
Szöveg	Kérj be egy mondatot, és számold meg, hány szóból áll és hány a betű van benne.

Téma	Feladat
Fájlkezelés 1	Kérj be 5 nevet, és írd ki őket a nevek.txt fájlba soronként. Olvasd vissza, és számozva írd ki.
Fájlkezelés 2	Készíts naplót : minden program-indításkor fűzz egy sort a naplo.txt -hez a dátummal. Írd ki, hányszor futott a program.
Fájlkezelés 3	A számkitalálós játék mentse el a legjobb eredményt (legkevesebb tipp) fájlba, és induláskor írja ki.
Osztály 1	Készíts Konyv osztályt (cím, szerző, év) konstruktorral és ToString -gel. Hozz létre 3 könyvet, és írd ki őket.
Osztály 2	Készíts Szamla osztályt Egyenleg tulajdonsággal, Befizet(int) és Kivesz(int) metódussal. Ne lehessen többet kivenni, mint amennyi van.
Öröklődés	Allat ősosztály Hangot_ad() virtual metódussal, és Kutya , Macska leszármazottak felülírt hanggal. Tedd őket egy List<Allat> -ba.
Nagy projekt	Könyvtár program : könyvek felvétele, listázás, keresés cím szerint, mentés és betöltés fájlból menüvel (do-while).

Mintamegoldás 1: FizzBuzz és prímszám

```
// 8. feladat: FizzBuzz
for (int i = 1; i <= 100; i++)
{
    if (i % 15 == 0)        Console.WriteLine("FizzBuzz");
    else if (i % 3 == 0)    Console.WriteLine("Fizz");
    else if (i % 5 == 0)    Console.WriteLine("Buzz");
    else                    Console.WriteLine(i);
}

// 13. feladat: prím-e?
static bool Prim(int n)
{
    if (n < 2) return false;
    for (int i = 2; i * i <= n; i++)
    {
        if (n % i == 0)
            return false;    // találtunk osztót: nem prím
    }
    return true;
}
```

- **FizzBuzz:** a 15-tel oszthatót kell előbb vizsgálni, mert az 3-mal és 5-tel is osztható. Különben sosem érnénk el.
- Az **else if** lánc az **első igaz ágon** megáll, ezért kell a sorrend.
- **Prím:** elég $\sqrt{n}$ -ig osztani ($i * i \leq n$), mert ha lenne nagyobb osztó, lenne hozzá kisebb is.
- A **return false** azonnal **kilép a metódusból és a ciklusból** is.

```
// 7. feladat: összeg, darabszám, átlag
int osszeg = 0, db = 0, szam;

Console.Write("Szám (0 = vége): ");
int.TryParse(Console.ReadLine(), out szam);

while (szam != 0)
{
    osszeg += szam;
    db++;
    Console.Write("Szám (0 = vége): ");
    int.TryParse(Console.ReadLine(), out szam);
}

if (db > 0)
{
    double atlag = (double)osszeg / db;
    Console.WriteLine($"Összeg: {osszeg}, átlag: {atlag}");
}

// 11. feladat: csillagpiramis
for (int sor = 1; sor <= 5; sor++)
{
    string szokoz = new string(' ', 5 - sor);
    string csillag = new string('*', 2 * sor - 1);
    Console.WriteLine(szokoz + csillag);
}
```

- **Átlag:** a beolvasás **kétszer** szerepel: a ciklus előtt (első szám) és a ciklus végén (következő szám). Ez a tipikus „beolvasás-ellenőrzés” minta.
- A **(double)osszeg / db** kell, különben egész osztás lenne (3 helyett 3,5).
- A **piramis:** a **sor.** sorban **5 - sor** szóköz és **2 * sor - 1** csillag van. Ezt is **táblázatba írva** érdemes kitalálni.
- Nem ugyanúgy kell megoldani! Ha működik és érted, jó a megoldásod.